

# Roteiro de Apresentação — Árvore B em Memória Secundária

**Duração alvo: 40 a 60 minutos** · 36 slides · ~1 a 1:30 por slide · AED-PG-2026

Como usar: cada slide tem a **mensagem-chave** (o que a banca tem que levar) e a **fala** (texto natural, pode adaptar). As frases em itálico são transições. No fim há um bloco de **perguntas prováveis**. >

**Público:** suponha que parte da plateia não conhece Árvore B. Não precisamos \*ensinar\* a estrutura, mas a linguagem deve ser acessível — fale em termos de "arquivo no disco" e "menos acessos = mais rápido". > **Ritmo:** 36 slides. A ~1 min/slide → ~40 min; a ~1:30/slide → ~54 min. Os slides marcados com (~) são candidatos a encurtar se o tempo apertar; os com (\*) merecem mais tempo.

| #  | Slide                                 | Alvo |
|----|---------------------------------------|------|
| 1  | Capa                                  | 1:00 |
| 2  | Roteiro da apresentação (~)           | 1:00 |
| 3  | O problema e os objetivos             | 1:30 |
| 4  | Contexto: memória × disco             | 1:30 |
| 5  | Contexto: índice e métrica honesta    | 1:30 |
| 6  | Anatomia de um nó (*)                 | 1:30 |
| 7  | Layout do arquivo no disco (*)        | 1:30 |
| 8  | Decisões de implementação             | 1:30 |
| 9  | O DiskManager (1 nó por I/O)          | 1:30 |
| 10 | Reaproveitamento: lista de livres     | 1:30 |
| 11 | Métodos: visão geral (~)              | 1:00 |
| 12 | Busca m-way                           | 1:30 |
| 13 | Inserção e split                      | 1:30 |
| 14 | Remoção                               | 1:30 |
| 15 | Resumo do que o código faz (~)        | 1:00 |
| 16 | Demonstração (GIF)                    | 1:30 |
| 17 | Grafos gerados                        | 1:00 |
| 18 | Os experimentos                       | 1:30 |
| 19 | Metodologia e ambiente                | 1:30 |
| 20 | Métricas                              | 1:00 |
| 21 | Impacto de m: I/O por busca (*)       | 2:00 |
| 22 | Impacto de m: altura                  | 1:30 |
| 23 | Resultados por operação               | 1:30 |
| 24 | Escala do conjunto (N)                | 1:30 |
| 25 | Reaproveitamento: resultado           | 1:30 |
| 26 | Dois sistemas: tempo (wall)           | 1:30 |
| 27 | Dois sistemas: CPU e determinismo (*) | 2:00 |
| 28 | Dois sistemas: validação (resumo) (~) | 1:00 |
| 29 | Dois sistemas: escala                 | 1:00 |
| 30 | E se usássemos union? (*)             | 2:00 |
| 31 | Dificuldades técnicas                 | 1:30 |

| #  | Slide                      | Alvo |
|----|----------------------------|------|
| 32 | Vantagens x desvantagens   | 1:30 |
| 33 | Aplicações                 | 1:00 |
| 34 | Ferramentas utilizadas     | 1:30 |
| 35 | Conclusão                  | 1:30 |
| 36 | Referências / encerramento | 0:30 |

## Parte 1 — A estrutura

**1. Capa.** Mensagem: o que é o trabalho em uma frase. "Nosso trabalho é uma **Árvore B de ordem m que opera estritamente em memória secundária** — a árvore vive num arquivo no disco e toda operação lê ou escreve **um nó por vez**, nunca a árvore inteira." Apresentem-se e citem o foco na avaliação experimental.

**2. Roteiro.** Mensagem: mapa da fala. "Primeiro a estrutura — contexto, como o nó e o arquivo são, os métodos e uma demo. Depois a avaliação — experimentos, impacto de m, comparação entre duas máquinas e a discussão final." Passe rápido.

**3. Problema e objetivos.** Mensagem: o que o enunciado pede. "Implementar a Árvore B residindo em **disco**, lida em blocos, **nunca inteira na RAM**; com busca, inserção e remoção; contando acessos ao disco. Acrescentamos uma avaliação empírica e validação em mais de uma máquina." "Por que disco importa tanto?"

**4. Contexto: memória x disco.** Mensagem: disco é o gargalo. "RAM é rápida, mas pequena e volátil; disco é enorme e permanente, porém **milhares de vezes mais lento**. Logo, o segredo da velocidade é fazer o **menor número de acessos ao disco**." "E como evitamos ler o arquivo todo?"

**5. Contexto: índice e métrica.** Mensagem: índice + métrica honesta. "Um **índice** diz onde cada registro está, evitando ler tudo — é o que a Árvore B faz em bancos de dados. E a métrica honesta **não é o relógio**, é o **número de acessos ao disco**, que independe do hardware." "Vamos ver como um nó é por dentro."

**\*\*6. Anatomia de um nó.** (\*) **Mensagem: o que há dentro de um nó.** "Cada nó é um registro de tamanho fixo: o campo n (quantas chaves), o vetor de ponteiros A[ ] (os RRN dos filhos) e o vetor de chaves K[ ]. Um nó de ordem m guarda até m-1 chaves. Ler um nó = um acesso ao disco\*\*." "E como esses nós ficam no arquivo?"

**\*\*7. Layout do arquivo.** (\*) **Mensagem: o arquivo é um vetor de registros.** "O arquivo é um vetor de registros **numerados por RRN**. O registro 0 é o header (raiz, total, free\_head). **Achamos qualquer nó por cálculo direto: offset = RRN x PAGE\_SIZE. E nós removidos entram numa** lista de livres\*\* para reuso." "Com isso, as decisões de projeto."

**8. Decisões de implementação.** Mensagem: as 4 decisões. "Ordem m como constante de compilação; raiz no header; **um nó por I/O**; e reaproveitamento de nós via lista de livres. À direita, uma árvore de ordem 3 real, exportada pelo código." "Toda a E/S passa por uma classe."

**9. O DiskManager.** Mensagem: onde mora o '1 nó por I/O'. "readNode e writeNode leem e escrevem **um** registro — e é aqui que fica o **contador de acessos**, a métrica central. A lógica da árvore só enxerga o disco pelo DiskManager, então nada vai inteiro para a RAM." "E quando um nó é liberado?"

**10. Reaproveitamento.** Mensagem: como a free list funciona. "Ao remover, o nó vira **livre** e entra numa pilha encadeada no próprio arquivo (cabeça no header). A próxima alocação **reusa** um livre antes de crescer o arquivo. É ligável — usamos isso para medir o ganho de espaço." "Agora os métodos."

**11. Métodos: visão geral. (~)** Mensagem: as três operações + ferramentas. "Busca, inserção e remoção, todas em disco, mais as ferramentas de monitoramento. Vou detalhar uma a uma." Não leia item a item.

**12. Busca m-way.** Mensagem: desce por faixas. "Em cada nó, as chaves dividem o universo em **faixas**; descemos pela faixa certa até a folha. O custo é **~a altura** em acessos — pouquíssimos. No grafo, buscar 70 custa 2 acessos." "Inserir é parecido, até estourar."

**13. Inserção e split.** Mensagem: split mantém o balanço. "Acha a folha e insere; se o nó **estoura**, dividimos em dois e a **mediana sobe** para o pai. Isso pode propagar e criar uma nova raiz — é o que mantém a árvore **sempre balanceada**."

"Remover é o caso mais delicado."

**14. Remoção.** Mensagem: sucessor, redistribuição, fusão. "Se a chave está num nó interno, trocamos pelo **sucessor**. Se uma folha fica abaixo do mínimo, reparamos:

**redistribuição** (empréstimo de um irmão) ou **fusão** (junta com o irmão). A fusão pode propagar para cima e baixar a árvore." "Num slide, tudo que o código faz."

**15. Resumo do código. (~)** Mensagem: núcleo + extras. "À esquerda o núcleo da árvore em disco; à direita os extras de avaliação (contador, free list, grafos, benchmark). O núcleo é a estrutura; os extras existem para **medi-la**." "Vamos ver rodando."

**16. Demonstração (GIF).** Mensagem: é real. "Execução de verdade: do make M=3 por **cada item do menu** — inserir, buscar, remover, acessos, imprimir, exportar grafo — e depois com m=5, onde a árvore fica mais rasa." "E o próprio programa desenha o grafo."

**17. Grafos gerados.** Mensagem: m maior = mais raso. "Exportados pelo programa via Graphviz. Cada caixa é um nó no disco (RRN). m=3 precisa de mais níveis; m=5 fica mais raso — **menos acessos**." "Como medimos isso de forma sistemática."

## Parte 2 — A avaliação

**18. Os experimentos.** Mensagem: a matriz cobre o enunciado. "Quatro experimentos: impacto de m; escala de N (mil a um milhão); ocupação com/sem reuso; e decomposição do tempo. Tudo aleatório e sequencial, em **duas máquinas**." "Como exatamente medimos."

**19. Metodologia e ambiente.** Mensagem: rigor. "Driver não interativo emite CSV; cada caso é reconstruído do zero; medimos com getrusage (CPU usuário, CPU sistema, espera de I/O); duas máquinas — Notebook e Titan/USP — com automação." "As seis métricas."

**20. Métricas. (~)** Mensagem: a honesta é o contador. "Acessos ao disco (a principal), altura, ocupação e tempo decomposto. A altura já cai em degraus com m."

"O resultado central: impacto de m."

**\*\*21. Impacto de m: I/O por busca. (\*) Mensagem: despenca e satura\*\*.** "Com m=3, 13 acessos por busca e 14 níveis. Subindo m, despenca: m=512 → 2 acessos. Mas o ganho

**satura** entre m=64 e 128 — a árvore já é rasa demais. Existe um **m ótimo**, do tamanho de um bloco de disco." "O mesmo aparece na altura."

**22. Impacto de m: altura.** Mensagem: altura ~ acessos. "A altura cai em degraus de 14 (m=3) para 2 (m grande). Altura baixa é a razão de existir da Árvore B." "E nas três operações?"

**23. Resultados por operação.** Mensagem: o ganho vale para todas. "Busca é a mais barata; inserção um pouco mais (split); remoção a mais cara (lê irmãos). As três caem com m e **todas saturam**." "E quando o conjunto cresce?"

**24. Escala do conjunto.** Mensagem: escalável. "Multiplicamos N por mil e o custo por busca **mal muda** — é logarítmico na base m. A estrutura aguenta bases enormes."

"O segundo eixo: espaço."

**25. Reaproveitamento: resultado.** Mensagem: ~30% de economia. "No churn (insere, remove metade, reinsere), com reuso o arquivo **não cresce**; sem reuso, **incha**. A free list economiza **27-30%**." "Sobre o processo: validação em duas máquinas."

**26. Dois sistemas: wall.** Mensagem: a forma é igual. "Notebook x Titan: acessos a disco **idênticos**; só o tempo muda. As curvas têm **a mesma forma**." "Por que o relógio engana — e a prova de determinismo."

**\*\*27. Dois sistemas: CPU e determinismo. (\*) Mensagem: idêntico nas duas. "À esquerda: o tempo é quase todo CPU de sistema (I/O via cache), por isso o relógio não é honesto. À direita: todos os pontos caem na reta  $y=x^{**}$  — acessos**

**rigorosamente iguais.** Determinístico e portátil." "Resumindo a validação."

**28. Validação: resumo. (~) Mensagem: portabilidade.** "Métricas de acesso idênticas byte a byte; só o tempo difere; comparação automatizada por script." "E a escala nas duas?"

**29. Dois sistemas: escala.** Mensagem: tendência igual. "De mil a um milhão nas duas máquinas: mesma tendência, Titan em paralelo abaixo. Escala comprovada." "Um ponto que o professor levantou: o union."

**\*\*30. E se usássemos union? (\*) Mensagem: página única x portabilidade. "Hoje header e nó são tipos separados, com serialização campo a campo. Com union\*\*, seriam uma**

**página única** de tamanho fixo: ler/gravar de uma vez, menos código e bugs, free list explícita.

Custo: **portabilidade** (padding/endianess) e o UB de type-punning, resolvível com memcopy/bit\_cast. Mais limpo, exigindo cuidado com o formato binário."

"Quais foram as dificuldades?"

**31. Dificuldades técnicas.** Mensagem: o que custou. "Indexação a partir do RRN 0; a ordem do path na remoção; o eofbit que silenciava leituras; e garantir 1 nó por I/O. A serialização verbosa foi o que motivou a ideia do union." "E os trade-offs?"

**32. Vantagens x desvantagens.** Mensagem: equilíbrio consciente. "Vantagens: rasa, balanceada, determinística, econômica. Custos: nós subocupados, remoção complexa, e m grande troca I/O por CPU — daí o **m ótimo**." "Onde isso é usado de verdade?"

**33. Aplicações.** Mensagem: está em tudo. "Índices de MySQL e PostgreSQL, sistemas de arquivos (NTFS, ext4), key-value stores (SQLite, LMDB). Eu usaria como índice de um banco em disco." "Com que ferramentas construímos isso?"

**34. Ferramentas utilizadas.** Mensagem: stack + papel da IA. "Implementação: C++17, Make, fstream, Graphviz. Avaliação: Python/matplotlib, Pillow (GIFs), python-pptx e reportlab (slides); Git; duas máquinas. E sou transparente: usamos o Claude como

**apoio, revisão e incremento** — no menu, na exportação de grafos e na revisão. A

**lógica da Árvore B é nossa**; a IA acelerou ferramentas acessórias." "Para fechar."

**35. Conclusão.** Mensagem: os três achados. "Um: I/O cai com m até **saturar** — existe m ótimo. Dois: a free list economiza ~**30%**. Três: resultados

**determinísticos** em duas máquinas. A Árvore B é rasa, balanceada e econômica — por isso é a base de bancos e sistemas de arquivos." "Referências e obrigado."

**36. Referências.** Comer (1979); Knuth, TAOCP vol. 3; Folk & Zoellick; Bayer & McCreight (1972); slides do Prof. Baranauskas. "Obrigado! Ficamos à disposição."

## Perguntas prováveis (preparação)

"**Árvore B não é binária?**" → "Não. Binária é a BST (2 filhos). A Árvore B é

**m-way** (até m filhos); o 'B' é de balanceada (Bayer). m grande deixa a árvore rasa. Ordem 2 é degenerada e o código a proíbe; ordem 3 é só o menor caso testado."

"**Por que o tempo aparece como CPU, não como espera de I/O?**" → "Porque writeNode faz flush para o **cache de páginas do SO**, não fsync no disco físico. O 'I/O' vira CPU de sistema. Por isso a métrica honesta é o **contador de acessos**."

"**O que mudaria de fato com o union no header?**" → "Header e nó virariam **uma página única** de tamanho fixo garantido em compilação: ler/gravar de uma vez, sem memcpy campo a campo, e free list explícita. Custo: portabilidade (padding/endianness) e UB de type-punning, resolvível com memcpy/std::bit\_cast."

"**Onde fica a raiz?**" → "No header, registro 0 — uma leitura de metadado e já sabemos a raiz."

"**Como funciona o reaproveitamento?**" → "Lista de livres encadeada no arquivo: cabeça em free\_head, cada livre guarda o próximo em K[1] com n=-1; allocNode puxa dela antes de estender o arquivo."

"**Qual o m ideal na prática?**" → "Aquele em que o nó preenche um **bloco de disco** (4-8 KB) — é onde a curva do slide 21 satura."

"**Por que a inserção sequencial gera arquivo maior?**" → "O split sempre cai à direita, deixando nós ~50% cheios; no aleatório a ocupação fica ~69% ( $\ln 2$ )."

"**A IA fez o trabalho?**" → "Não. A lógica da Árvore B é nossa; a IA apoiou ferramentas acessórias (menu, grafos, scripts) e revisão. As decisões e a análise são nossas."

---

## Dicas de cronometragem

- **40 min (rápido):** ~1 min/slide; encurte os marcados (~) (2, 11, 15, 20, 28) lendo só os títulos.
- **50-60 min (completo):** detalhe os marcados (\*) (6, 7, 21, 27, 30) e responda perguntas no fim de cada parte.
- **Não leia os slides palavra por palavra** — a fala acima é a narração; os slides são o apoio visual.